

LLD Complete Reference

Low Level Design — SDE2 FAANG Level

OOP | SOLID | 13 Patterns | Concurrency Deep Dive | 9 Problems | Protocol

Part 1: OOP Fundamentals — 4 Pillars with Java examples

Part 2: SOLID Principles — Bad + Good code for each

Part 3: Creational Patterns — Singleton (DCL), Factory, Builder, Prototype

Part 4: Structural Patterns — Adapter, Decorator, Facade, Proxy

Part 5: Behavioural Patterns — Strategy, Observer, Command, Template, State, Chain

Part 6: Concurrency Deep Dive — volatile, Lock types, ReadWriteLock, Producer-Consumer, LRU

Part 7: Problem 1 — Parking Lot (Singleton + Strategy + thread safety)

Part 8: Problem 2 — Rate Limiter + Distributed extension (Redis)

Part 9: Problem 3 — Splitwise (min-transactions settlement algorithm)

Part 10: Problem 4 — Elevator System (SCAN + State machine)

Part 11: Problem 5 — BookMyShow (concurrent seat locking + timeout)

Part 12: Problem 6 — Snake and Ladder (clean entity design)

Part 13: Problem 7 — Vending Machine (State pattern deep)

Part 14: Problem 8 — ATM System (concurrent withdrawal + rollback)

Part 15: Problem 9 — Library Management (Observer + Strategy + fine)

Part 16: Interview Protocol — 45-min framework, timing guide, scoring rubric

Java | Target: 7/10 LLD | All 9 problems implementable in 40 min from blank file

Part 1: OOP Fundamentals

The 4 pillars. Know as justifications for every design decision.

1. Encapsulation — Hide state, expose behaviour

```
public class BankAccount {

    private double balance; // private -- not directly settable

    public void deposit(double amount) {

        if (amount <= 0) throw new IllegalArgumentException("Must be positive");

        this.balance += amount; // validation lives HERE not in caller

    }

    public void withdraw(double amount) {

        if (amount > balance) throw new InsufficientFundsException();

        this.balance -= amount;

    }

    public double getBalance() { return balance; } // read-only

}

// Caller cannot do: account.balance = -1000. Must go through validated methods.
```

2. Abstraction — Hide implementation, expose contract

```
public interface PaymentProcessor {

    boolean processPayment(double amount, String currency);

    void refund(String transactionId);

}

public class OrderService {

    private final PaymentProcessor processor; // abstraction

    public OrderService(PaymentProcessor p) { this.processor = p; }

    public void placeOrder(Order o) { processor.processPayment(o.getTotal(),"INR"); }

}

// Swap Razorpay for Stripe: new StripeProcessor() injected. Zero OrderService change.
```

3. Inheritance — IS-A relationship

```
public abstract class Vehicle {

    protected String licensePlate;
```

```

public abstract String getType();

public abstract int getCapacity();

}

public class Car extends Vehicle { public String getType(){return "CAR";} public int getCapacity(){return 4;} }

public class Truck extends Vehicle { public String getType(){return "TRUCK";} public int getCapacity(){return 2;} }

```

4. Polymorphism — Same interface, different behaviour

```

List<Vehicle> vehicles = List.of(new Car("KA01"), new Truck("MH02"));

for (Vehicle v : vehicles) {

    System.out.println(v.getType() + " capacity: " + v.getCapacity());

} // CAR capacity: 4 / TRUCK capacity: 2 -- same call, different behaviour

```

Composition vs Inheritance

Criterion	Inheritance	Composition
Relationship	IS-A (Car IS-A Vehicle)	HAS-A (Car HAS-A Engine)
Flexibility	Low — tight coupling to parent	High — swap implementation at runtime
Testability	Hard — must test through parent	Easy — mock the composed object
Default choice	Use rarely, only true IS-A	Always prefer this

Part 2: SOLID Principles

For each principle: write the BAD version first, then the GOOD version.

S — Single Responsibility: one class, one reason to change

```
// BAD: Order class does everything

public class Order {

    public void calculateTotal() { }

    public void saveToDatabase() { } // NOT Order responsibility

    public void sendEmail() { } // NOT Order responsibility

}

// GOOD: separate classes, each one reason to change

public class Order { public void calculateTotal() { } }

public class OrderRepository { public void save(Order o) { } }

public class OrderNotifier { public void sendEmail(Order o) { } }
```

O — Open/Closed: extend without modifying existing code

```
// BAD: modify existing code for every new payment type

if (type.equals("upi")) { } else if (type.equals("card")) { } // grow forever

// GOOD: new payment = new class only

public interface PaymentStrategy { void process(double amount); }

public class UpiPayment implements PaymentStrategy { public void process(double a){} }

public class CardPayment implements PaymentStrategy { public void process(double a){} }

// Adding BNPL: just new BNPLPayment class. Zero existing changes.
```

L — Liskov Substitution: subclass must not break parent contract

```
// BAD: Penguin cannot fly -- breaks Bird contract

class Bird { public void fly() { } }

class Penguin extends Bird { public void fly() { throw new UnsupportedOperationException(); } }

// GOOD: segregate interface so Penguin never promises to fly

interface Bird { String getName(); }

interface FlyingBird extends Bird { void fly(); }

class Eagle implements FlyingBird { public void fly() { } }

class Penguin implements Bird { /* no fly method */ }
```

I — Interface Segregation: no forced implementation of unused methods

```
// BAD: Robot forced to implement eat() and sleep()

interface Worker { void work(); void eat(); void sleep(); }

// GOOD: split into focused interfaces

interface Workable { void work(); }

interface Eatable { void eat(); }

class Human implements Workable, Eatable { }

class Robot implements Workable { } // clean -- no fake methods
```

D — Dependency Inversion: depend on abstractions not concretions

```
// BAD: hardcoded dependency -- cannot swap without rewriting

class NotificationService { private EmailSender sender = new EmailSender(); }

// GOOD: inject the abstraction

interface MessageSender { void send(String recipient, String msg); }

class NotificationService {

    private final MessageSender sender;

    public NotificationService(MessageSender s) { this.sender = s; }

    public void notify(String u, String m) { sender.send(u, m); }

}

// Switch Email to SMS: inject new SmsSender(). Zero NotificationService change.
```

Part 3: Creational Patterns

How objects are created. Name the pattern out loud as you use it.

Singleton — One instance globally (thread-safe DCL)

```
public class DatabaseConnection {

    private static volatile DatabaseConnection instance; // volatile = mandatory

    private DatabaseConnection() { /* expensive init */ }

    public static DatabaseConnection getInstance() {

        if (instance == null) { // check 1: no lock (fast path)

            synchronized (DatabaseConnection.class) {

                if (instance == null) { // check 2: under lock (safe)

                    instance = new DatabaseConnection();

                }

            }

        }

        return instance;

    }

}

// volatile prevents JIT reorder: alloc->ASSIGN ref->init

// Without volatile: Thread B sees non-null reference but uninitialised object = crash
```

Factory — Centralise creation, caller does not know concrete class

```
public interface Notification { void send(String msg); }

public class PushNotification implements Notification { public void send(String m){} }

public class SmsNotification implements Notification { public void send(String m){} }

public class EmailNotification implements Notification { public void send(String m){} }

public class NotificationFactory {

    public static Notification create(String channel) {

        return switch (channel.toUpperCase()) {

            case "PUSH" -> new PushNotification();

            case "SMS" -> new SmsNotification();

            case "EMAIL" -> new EmailNotification();

        }

    }

}
```

```
default -> throw new IllegalArgumentException("Unknown: " + channel);

};

}

}
```

Builder — Complex objects with many optional fields

```
public class Order {

    private final String userId; private final List<Item> items; // required

    private final String coupon; private final String notes; // optional

    private Order(Builder b) { userId=b.userId; items=b.items; coupon=b.coupon; notes=b.notes; }

    public static class Builder {

        private final String userId; private final List<Item> items;

        private String coupon; private String notes;

        public Builder(String u, List<Item> i) {

            if(u==null||i==null) throw new IllegalArgumentException("Required");

            userId=u; items=i;

        }

        public Builder coupon(String c) { coupon=c; return this; }

        public Builder notes(String n) { notes=n; return this; }

        public Order build() { return new Order(this); }

    }

}

// new Order.Builder("u1",items).coupon("SAVE10").notes("Ring bell").build()
```

Part 4: Structural Patterns

How classes compose into larger structures.

Decorator — Stack features at runtime without modifying original class

```
public interface Coffee { double getCost(); String getDescription(); }

public class SimpleCoffee implements Coffee {

    public double getCost() { return 50.0; }

    public String getDescription() { return "Coffee"; }

}

public abstract class CoffeeDecorator implements Coffee {

    protected final Coffee coffee;

    public CoffeeDecorator(Coffee c) { this.coffee = c; }

}

public class MilkDecorator extends CoffeeDecorator {

    public MilkDecorator(Coffee c) { super(c); }

    public double getCost() { return coffee.getCost() + 15.0; }

    public String getDescription() { return coffee.getDescription() + ", Milk"; }

}

public class SugarDecorator extends CoffeeDecorator {

    public SugarDecorator(Coffee c) { super(c); }

    public double getCost() { return coffee.getCost() + 5.0; }

    public String getDescription() { return coffee.getDescription() + ", Sugar"; }

}

// Stack: new SugarDecorator(new MilkDecorator(new SimpleCoffee())) = Rs.70, "Coffee, Milk, Sugar"
```

Adapter — Wrap incompatible interface

```
public interface ModernPayment { void pay(String userId, double amount); }

public class LegacyPayment { public boolean makePayment(int id, float amt){return true;} }

public class PaymentAdapter implements ModernPayment {

    private final LegacyPayment legacy;

    public PaymentAdapter(LegacyPayment l) { this.legacy = l; }

    public void pay(String userId, double amount) {
```

```

    legacy.makePayment(Integer.parseInt(userId), (float)amount);

}

}

```

Facade — Simple interface over complex subsystem

```

public class OrderFacade {

    private final PaymentGateway payment = new PaymentGateway();

    private final InventorySystem inventory = new InventorySystem();

    private final EmailService email = new EmailService();

    public void placeOrder(Order order) {

        payment.charge(order.getCard(), order.getTotal());

        inventory.reserve(order.getItemId());

        email.sendConfirmation(order.getEmail());

    }

}

// Caller: orderFacade.placeOrder(order); -- one line hides all complexity

```

Proxy — Control access (cache, lazy load, access control)

```

public interface DataService { List<User> getUsers(); }

public class RealDataService implements DataService {

    public List<User> getUsers() { return database.findAll(); } // expensive

}

public class CachingProxy implements DataService {

    private final DataService real; private List<User> cache; private long cacheTime;

    private static final long TTL = 60_000;

    public CachingProxy(DataService r) { this.real = r; }

    public List<User> getUsers() {

        long now = System.currentTimeMillis();

        if (cache == null || now - cacheTime > TTL) { cache=real.getUsers(); cacheTime=now; }

        return cache;

    }

}

```

Part 5: Behavioural Patterns

How objects interact and communicate.

Strategy — Swap algorithms at runtime

```
public interface SortStrategy { void sort(int[] arr); }

public class QuickSort implements SortStrategy { public void sort(int[] a) { /*qsort*/ } }

public class MergeSort implements SortStrategy { public void sort(int[] a) { /*msort*/ } }

public class Sorter {

    private SortStrategy strategy;

    public Sorter(SortStrategy s) { this.strategy = s; }

    public void setStrategy(SortStrategy s) { this.strategy = s; }

    public void sort(int[] arr) { strategy.sort(arr); }

}

// sorter.setStrategy(new MergeSort()); -- swap at runtime, Sorter unchanged
```

Observer — Event-driven decoupled notification

```
public interface Observer { void update(String eventType, Object data); }

public class EventBus {

    private final Map<String, List<Observer>> listeners = new HashMap<>();

    public void subscribe(String event, Observer o) {

        listeners.computeIfAbsent(event, k -> new ArrayList<>()).add(o);

    }

    public void unsubscribe(String event, Observer o) {

        listeners.getOrDefault(event, List.of()).remove(o);

    }

    public void publish(String event, Object data) {

        listeners.getOrDefault(event, List.of()).forEach(o -> o.update(event, data));

    }

}

// bus.subscribe("ORDER_PLACED", (t,d) -> emailService.send(d));

// bus.publish("ORDER_PLACED", order); // notifies all subscribers
```

Command — Encapsulate operation as object (undo/redo)

```

public interface Command { void execute(); void undo(); }

public class InsertCommand implements Command {

    private final TextEditor editor; private final String text; private final int pos;

    public InsertCommand(TextEditor e,String t,int p){editor=e;text=t;pos=p;}

    public void execute() { editor.insertText(text, pos); }

    public void undo() { editor.deleteText(pos, text.length()); }

}

public class CommandHistory {

    private final Deque<Command> history = new ArrayDeque<>();

    public void execute(Command cmd) { cmd.execute(); history.push(cmd); }

    public void undo() { if (!history.isEmpty()) history.pop().undo(); }

}

```

Template Method — Fixed skeleton, varying steps

```

public abstract class DataProcessor {

    public final void process() { // FINAL -- skeleton cannot be overridden

        readData(); validateData(); processData(); writeData(); generateReport();

    }

    protected abstract void readData();

    protected abstract void processData();

    protected abstract void writeData();

    protected void validateData() { System.out.println("Default validation"); }

    protected void generateReport() { System.out.println("Default report"); }

}

public class CSVProcessor extends DataProcessor {

    protected void readData() { /* read CSV */ }

    protected void processData() { /* parse rows */ }

    protected void writeData() { /* write DB */ }

}

```

State — Object changes behaviour based on state

```

public interface ElevatorState {

    void requestFloor(Elevator ctx, int floor);

}

```

```

String getStateName();

}

public class IdleState implements ElevatorState {

    public void requestFloor(Elevator ctx, int floor) {

        System.out.println("Moving to floor " + floor);

        ctx.setState(new MovingState()); // transition

    }

    public String getStateName() { return "IDLE"; }

}

public class MovingState implements ElevatorState {

    public void requestFloor(Elevator ctx, int floor) {

        System.out.println("Already moving, queuing floor " + floor);

    }

    public String getStateName() { return "MOVING"; }

}

public class Elevator {

    private ElevatorState state = new IdleState();

    public void setState(ElevatorState s) { this.state = s; }

    public void requestFloor(int floor) { state.requestFloor(this, floor); }

}

```

Chain of Responsibility — Pass request along handler chain

```

public abstract class SupportHandler {

    protected SupportHandler next;

    public SupportHandler setNext(SupportHandler n) { next = n; return n; }

    public abstract void handle(Ticket ticket);

    protected void passToNext(Ticket t) {

        if (next != null) next.handle(t);

        else System.out.println("Unresolved: " + t.getId());

    }

}

public class L1Support extends SupportHandler {

```


Part 6: Concurrency Deep Dive

Most missed topic in LLD interviews. Know when to use each primitive.

volatile vs synchronized

```
// volatile: visibility + ordering, NOT atomicity

volatile boolean shutdown = false;

// Writer: shutdown = true; -- immediately visible to all threads

// Reader: while (!shutdown) { doWork(); } -- sees update immediately


// volatile FAILS for compound operations:

volatile int count = 0;

count++; // NOT atomic: read + increment + write = 3 steps = RACE CONDITION


// synchronized: visibility + ordering + mutual exclusion

public synchronized void increment() { count++; } // compound action, atomic


// Rule: single variable, one writer = volatile OK

// Rule: compound operations (check-then-act) = synchronized or AtomicInteger
```

Lock Types — When to Use Which

Lock	Use When	Key Feature
synchronized	Simple mutual exclusion	Simplest. Cannot timeout. Reentrant.
ReentrantLock	Need tryLock or lockInterruptibly	Must unlock in finally. Can timeout.
ReadWriteLock	Many readers, few writers	Multiple readers OR one writer.
AtomicInteger	Single variable high-contention counter	Lock-free CAS. Faster than synchronized.
StampedLock	Optimistic reads (Java 8+)	Fastest reads. Complex to use.

ReentrantReadWriteLock — Reader-Writer Problem

```
public class ThreadSafeCache<K,V> {

    private final Map<K,V> cache = new HashMap<>();

    private final ReadWriteLock lock = new ReentrantReadWriteLock();

    private final Lock readLock = lock.readLock();

    private final Lock writeLock = lock.writeLock();

}
```

```

public V get(K key) {

    readLock.lock(); // multiple threads can read simultaneously

    try { return cache.get(key); }

    finally { readLock.unlock(); }

}

public void put(K key, V value) {

    writeLock.lock(); // exclusive

    try { cache.put(key, value); }

    finally { writeLock.unlock(); }

}

}

// Use when: reads >> writes -- leaderboard, config cache, catalog

```

Thread-Safe LRU Cache

```

public class LRUCache<K,V> {

    private final int capacity;

    private final LinkedHashMap<K,V> cache;

    private final ReadWriteLock lock = new ReentrantReadWriteLock();

    public LRUCache(int cap) {

        this.capacity = cap;

        this.cache = new LinkedHashMap<K,V>(cap, 0.75f, true) { // accessOrder=true

            protected boolean removeEldestEntry(Map.Entry<K,V> e) {

                return size() > capacity; // evict oldest when full

            }

        };

    }

    public V get(K key) {

        lock.readLock().lock();

        try { return cache.getDefault(key, null); }

        finally { lock.readLock().unlock(); }

    }

}

```

```

public void put(K key, V value) {

    lock.writeLock().lock();

    try { cache.put(key, value); }

    finally { lock.writeLock().unlock(); }

}

}

```

Producer-Consumer — BlockingQueue (simple) and Lock+Condition (deep)

```

// Option 1: BlockingQueue -- production-ready, use this first

public class TaskQueue {

    private final BlockingQueue<Task> queue;

    public TaskQueue(int cap) { queue = new LinkedBlockingQueue<>(cap); }

    public void produce(Task t) throws InterruptedException { queue.put(t); }

    public Task consume() throws InterruptedException { return queue.take(); }

}

// Option 2: Lock + Condition -- shows depth when interviewer asks "how does BlockingQueue work?"

public class BoundedBuffer<T> {

    private final Queue<T> buffer = new LinkedList<>();

    private final int capacity;

    private final Lock lock = new ReentrantLock();

    private final Condition notFull = lock.newCondition();

    private final Condition notEmpty = lock.newCondition();

    public BoundedBuffer(int cap) { this.capacity = cap; }

    public void put(T item) throws InterruptedException {

        lock.lock();

        try {

            while (buffer.size() == capacity) notFull.await();

            buffer.offer(item);

            notEmpty.signal(); // wake one waiting consumer

        } finally { lock.unlock(); }

    }

}

```

```

public T take() throws InterruptedException {

    lock.lock();

    try {

        while (buffer.isEmpty()) notEmpty.await();

        T item = buffer.poll();

        notFull.signal(); // wake one waiting producer

        return item;

    } finally { lock.unlock(); }

}

}

// Interview: mention BlockingQueue first. Offer Lock+Condition when asked "how does it work?"

```

ThreadLocal — Per-Thread State

```

ThreadLocal<Connection> conn = ThreadLocal.withInitial(() -> createConn());

// MEMORY LEAK in thread pools: threads reused, ThreadLocal persists

try { conn.get().executeQuery("..."); }

finally { conn.remove(); } // MANDATORY -- prevents data leaking between requests

```

Concurrency Decision Guide

Scenario	Tool	Why
Simple flag one writer	volatile boolean	No compound operation
Counter many threads	AtomicInteger	Lock-free CAS
Check-then-act	synchronized	Compound action needs mutex
Many readers few writers	ReadWriteLock	Concurrent reads
Producer-consumer	BlockingQueue	Built-in blocking
Per-request context	ThreadLocal	Per-thread isolation, remove in finally
Single instance	DCL with volatile	Fast path + safe init

Part 7: Problem 1 — Parking Lot

Singleton + Strategy + thread safety. Most frequently asked.

Clarify: floors? vehicle types? pricing? concurrent? display?

Patterns: Singleton (ParkingLot), Strategy (Pricing), Factory (optional)

```
public enum VehicleType { BIKE, CAR, TRUCK }

public enum SpotType { SMALL, MEDIUM, LARGE }

public enum SpotStatus { AVAILABLE, OCCUPIED }

public abstract class Vehicle {

    protected final String plate; protected final VehicleType type;

    public Vehicle(String p,VehicleType t){plate=p;type=t;}

    public VehicleType getType(){return type;}

}

public class Bike extends Vehicle{public Bike(String p) {super(p,VehicleType.BIKE);}}

public class Car extends Vehicle{public Car(String p) {super(p,VehicleType.CAR);}}

public class Truck extends Vehicle{public Truck(String p){super(p,VehicleType.TRUCK);}}

public class ParkingSpot {

    private final String id; private final SpotType type;

    private SpotStatus status=SpotStatus.AVAILABLE; private Vehicle parked;

    public ParkingSpot(String id,SpotType t){this.id=id;this.type=t;}

    public synchronized boolean isAvailable(){return status==SpotStatus.AVAILABLE;}

    public synchronized void park(Vehicle v){

        if(!isAvailable()) throw new IllegalStateException("Spot occupied");

        parked=v; status=SpotStatus.OCCUPIED;

    }

    public synchronized void vacate(){parked=null; status=SpotStatus.AVAILABLE;}

    public SpotType getType(){return type;} public String getId(){return id;}

}

public class Floor {

    private final List<ParkingSpot> spots;

    public Floor(List<ParkingSpot> s){spots=s;}
```



```

private ParkingLot(List<Floor> f,PricingStrategy p){floors=f;pricing=p;}

public static ParkingLot getInstance(List<Floor> f,PricingStrategy p){

if(instance==null){synchronized(ParkingLot.class){if(instance==null)instance=new ParkingLot(f,p);}}

return instance;

}

public Ticket park(Vehicle v){

ParkingSpot spot=floors.stream()

.map(f->f.findAvailable(v.getType())).filter(Optional::isPresent)

.map(Optional::get).findFirst()

.orElseThrow(()->new RuntimeException("Full"));

spot.park(v);

Ticket t=new Ticket(v,spot); tickets.put(t.getTicketId(),t); return t;

}

public double exit(String ticketId){

Ticket t=tickets.remove(ticketId);

if(t==null) throw new IllegalArgumentException("Invalid ticket");

t.markExit(); t.getSpot().vacate();

return pricing.calculate(t.getDurationHours(),t.getVehicle().getType());

}

}

```

- + Singleton: ParkingLot with DCL + volatile
- + Strategy: PricingStrategy — swap pricing without touching ParkingLot
- + synchronized on ParkingSpot.park/vacate — thread-safe spot reservation
- + ConcurrentHashMap for activeTickets — thread-safe ticket tracking

Part 8: Problem 2 — Rate Limiter + Distributed Extension

Strategy + Factory + thread safety. Rubrik/Rippling favourite.

Patterns: Strategy (Algorithm), Factory, Facade

```
public enum RateLimitAlgo { TOKEN_BUCKET, SLIDING_WINDOW }

public enum RateLimitResult { ALLOWED, DENIED }

public class RateLimitConfig {

    private final int maxRequests; private final long windowMs; private final RateLimitAlgo strategy;

    public RateLimitConfig(int r, long w, RateLimitAlgo s){maxRequests=r;windowMs=w;strategy=s;}

    public int getMaxRequests(){return maxRequests;}

    public long getWindowMs(){return windowMs;}

    public RateLimitAlgo getStrategy(){return strategy;}

}

public interface RateLimitAlgorithm {

    RateLimitResult tryAcquire(String clientId);

    int getRemaining(String clientId);

}

public class TokenBucketAlgorithm implements RateLimitAlgorithm {

    private static class Bucket { volatile double tokens; volatile long lastRefill;

        Bucket(double t, long ts){tokens=t;lastRefill=ts;} }

    private final int capacity; private final double refillPerMs;

    private final ConcurrentHashMap<String, Bucket> buckets=new ConcurrentHashMap<>();

    public TokenBucketAlgorithm(RateLimitConfig c){

        capacity=c.getMaxRequests(); refillPerMs=(double)capacity/c.getWindowMs(); }

    public synchronized RateLimitResult tryAcquire(String clientId){

        long now=System.currentTimeMillis();

        Bucket b=buckets.computeIfAbsent(clientId, k->new Bucket(capacity, now));

        b.tokens=Math.min(capacity, b.tokens+(now-b.lastRefill)*refillPerMs);

        b.lastRefill=now;

        if(b.tokens>=1.0){b.tokens-=1.0;return RateLimitResult.ALLOWED;}

        return RateLimitResult.DENIED;

    }

}
```


Part 9: Problem 3 — Splitwise

Template Method (Split types), greedy settlement algorithm.

```
public abstract class Split {

    protected final User user; protected double amount;

    public Split(User u){user=u;}

    public abstract double calculateAmount(double total);

    public User getUser(){return user;} public double getAmount(){return amount;}

}

public class EqualSplit extends Split {

    private final int n;

    public EqualSplit(User u,int n){super(u);this.n=n;}

    public double calculateAmount(double total){amount=total/n;return amount;}

}

public class ExactSplit extends Split {

    public ExactSplit(User u,double amt){super(u);amount=amt;}

    public double calculateAmount(double total){return amount;}

}

public class PercentageSplit extends Split {

    private final double pct;

    public PercentageSplit(User u,double p){super(u);pct=p;}

    public double calculateAmount(double total){amount=total*pct/100;return amount;}

}

public class Expense {

    private final String desc; private final double total;

    private final User paidBy; private final List<Split> splits;

    public Expense(String d,double t,User p,List<Split> s){

        desc=d;total=t;paidBy=p;splits=s;

        double sum=s.stream().mapToDouble(sp->sp.calculateAmount(t)).sum();

        if(Math.abs(sum-t)>0.01) throw new IllegalArgumentException("Splits dont add up to total");

    }

    public User getPaidBy(){return paidBy;} public List<Split> getSplits(){return splits;}
```


Part 10: Problem 4 — Elevator System

SCAN algorithm + State machine + Dispatching strategy.

```
public enum Direction { UP, DOWN, IDLE }

public enum ElevatorStatus { MOVING, STOPPED, MAINTENANCE }

public class Elevator {

    private final int id; private volatile int currentFloor;

    private volatile Direction direction=Direction.IDLE;

    private volatile ElevatorStatus status=ElevatorStatus.STOPPED;

    private final TreeSet<Integer> upReqs=new TreeSet<>();

    private final TreeSet<Integer> downReqs=new TreeSet<>(Comparator.reverseOrder());

    private final int totalFloors;

    public Elevator(int id,int start,int floors){this.id=id;currentFloor=start;totalFloors=floors;}

    public synchronized void addRequest(int floor){

        if(floor>currentFloor) upReqs.add(floor);

        else if(floor<currentFloor) downReqs.add(floor);

    }

    // SCAN: serve all in one direction, then reverse

    public synchronized int getNextFloor(){

        if(direction==Direction.UP||direction==Direction.IDLE){

            if(!upReqs.isEmpty()){direction=Direction.UP;return upReqs.first();}

            if(!downReqs.isEmpty()){direction=Direction.DOWN;return downReqs.first();}

        } else {

            if(!downReqs.isEmpty()){direction=Direction.DOWN;return downReqs.first();}

            if(!upReqs.isEmpty()){direction=Direction.UP;return upReqs.first();}

        }

        direction=Direction.IDLE; return currentFloor;

    }

    public synchronized void moveToFloor(int f){

        status=ElevatorStatus.MOVING; currentFloor=f;

        upReqs.remove(f); downReqs.remove(f); status=ElevatorStatus.STOPPED;

    }

}
```


Part 11: Problem 5 — BookMyShow

Concurrent seat locking + state machine + atomic-or-rollback.

```
public enum SeatType { REGULAR, PREMIUM, VIP }

public enum SeatStatus { AVAILABLE, RESERVED, BOOKED }

public enum BookingStatus { PENDING, CONFIRMED, CANCELLED, EXPIRED }

public class Seat {

    private final String seatId; private final SeatType type;

    private volatile SeatStatus status=SeatStatus.AVAILABLE;

    private volatile String reservedBy; private volatile LocalDateTime reservedAt;

    private static final int TIMEOUT_MIN=10;

    public Seat(String id,SeatType t){seatId=id;type=t;}

    public synchronized boolean reserve(String bookingId){

        if(status==SeatStatus.BOOKED) return false;

        if(status==SeatStatus.RESERVED&&!isExpired()) return false;

        status=SeatStatus.RESERVED; reservedBy=bookingId; reservedAt=LocalDateTime.now();

        return true;

    }

    public synchronized boolean confirm(String bookingId){

        if(status!=SeatStatus.RESERVED||!bookingId.equals(reservedBy)) return false;

        if(isExpired()){release();return false;}

        status=SeatStatus.BOOKED; return true;

    }

    public synchronized void release(){status=SeatStatus.AVAILABLE;reservedBy=null;reservedAt=null;}

    private boolean isExpired(){return
        reservedAt!=null&&LocalDateTime.now().isAfter(reservedAt.plusMinutes(TIMEOUT_MIN));}

    public SeatStatus getStatus(){return status;} public SeatType getType(){return type;}

    public String getSeatId(){return seatId;}

}

public class Booking {

    private final String bookingId=UUID.randomUUID().toString();

    private final User user; private final Show show; private final List<Seat> seats;
```


Part 12: Problem 6 — Snake and Ladder

Clean entity design, Template Method (Cell), circular queue.

```
public class Dice{

    private final int sides; private final Random r=new Random();

    public Dice(int s){if(s<2)throw new IllegalArgumentException(">=2 sides required");sides=s;}

    public int roll(){return r.nextInt(sides)+1;}

}

public abstract class Cell{

    protected final int position;

    public Cell(int p){position=p;}

    public abstract int getEndPosition();

    public abstract String getType();

}

public class Snake extends Cell{

    private final int tail;

    public Snake(int head,int tail){super(head);

    if(tail>=head)throw new IllegalArgumentException("Tail below head required");this.tail=tail;}

    public int getEndPosition(){return tail;} public String getType(){return "Snake";}

}

public class Ladder extends Cell{

    private final int top;

    public Ladder(int bottom,int top){super(bottom);

    if(top<=bottom)throw new IllegalArgumentException("Top above bottom required");this.top=top;}

    public int getEndPosition(){return top;} public String getType(){return "Ladder";}

}

public class Player{

    private final String name; private int position=0;

    public Player(String n){name=n;}

    public void moveTo(int p){position=p;}

    public int getPosition(){return position;} public String getName(){return name;}

}
```


Part 13: Problem 7 — Vending Machine

State pattern deep. Each state handles all events differently.

Patterns: State (machine behaviour), zero if/else chains anywhere

```
public class Item{

    private final String name; private final double price; private int qty;

    public Item(String n,double p,int q){name=n;price=p;qty=q;}

    public boolean isAvailable(){return qty>0;}

    public void dispense(){if(!isAvailable())throw new RuntimeException("Out of stock");qty--;}

    public double getPrice(){return price;} public String getName(){return name;}

}

public interface State{

    void insertCoin(VendingMachine vm,double amount);

    void selectItem(VendingMachine vm,String code);

    void dispense(VendingMachine vm);

    void refund(VendingMachine vm);

}

public class IdleState implements State{

    public void insertCoin(VendingMachine vm,double amount){

        vm.addBalance(amount); System.out.println("Inserted Rs."+amount);

        vm.setState(new HasMoneyState());

    }

    public void selectItem(VendingMachine vm,String c){System.out.println("Insert coin first");}

    public void dispense(VendingMachine vm){System.out.println("No item selected");}

    public void refund(VendingMachine vm){System.out.println("Nothing to refund");}

}

public class HasMoneyState implements State{

    public void insertCoin(VendingMachine vm,double a){vm.addBalance(a);System.out.println("Balance: Rs."+vm.getBalance());}

    public void selectItem(VendingMachine vm,String code){

        Item item=vm.getItem(code);

        if(item==null){System.out.println("Not found");return;}

        if(!item.isAvailable()){vm.setState(new OutOfStockState());return;}

    }

}
```



```

private double balance=0; private String selected=null;

private final Map<String,Item> inventory=new HashMap<>();

public void addItem(String code,Item item){inventory.put(code,item);}

public void insertCoin(double a){state.insertCoin(this,a);}

public void selectItem(String code){state.selectItem(this,code);}

public void dispenseItem(){state.dispense(this);}

public void refund(){state.refund(this);}

public void setState(State s){state=s;}

public void addBalance(double a){balance+=a;} public void setBalance(double b){balance=b;}

public double getBalance(){return balance;}

public Item getItem(String c){return inventory.get(c);}

public void setSelected(String c){selected=c;} public String getSelected(){return selected;}

}

```

- + State: no if/else chains anywhere — behaviour fully in state classes
- + Extensible: add MaintenanceState = new class, zero existing changes

Part 14: Problem 8 — ATM System

Concurrent withdrawal + state machine + compensating transaction.

```
public enum ATMState { IDLE, CARD_INSERTED, AUTHENTICATED, DISPENSING }

public enum TransactionType { WITHDRAWAL, DEPOSIT, BALANCE_CHECK }

public enum TxnStatus { SUCCESS, FAILED, INSUFFICIENT_FUNDS }

public class Account{

    private final String id; private double balance; private final String pin;

    public Account(String id,double bal,String pin){this.id=id;balance=bal;this.pin=pin;}

    public synchronized TxnStatus withdraw(double amount){

        if(amount<=0) return TxnStatus.FAILED;

        if(amount>balance) return TxnStatus.INSUFFICIENT_FUNDS;

        balance-=amount; return TxnStatus.SUCCESS;

    }

    public synchronized void deposit(double a){if(a>0)balance+=a;}

    public synchronized double getBalance(){return balance;}

    public boolean verifyPin(String p){return pin.equals(p);}

    public String getId(){return id;}

}

public class CashDispenser{

    private int cash;

    public CashDispenser(int c){cash=c;}

    public synchronized boolean dispense(int amount){

        if(amount>cash){System.out.println("ATM out of cash");return false;}

        cash-=amount; System.out.println("Dispensing Rs."+amount); return true;

    }

}

public class ATM{

    private ATMState state=ATMState.IDLE;

    private Account current=null;

    private final Map<String,Account> accounts;
```


Part 15: Problem 9 — Library Management

Observer + Strategy + fine calculation.

```
public enum BookStatus { AVAILABLE, CHECKED_OUT, RESERVED, LOST }

public enum MemberType { STUDENT, FACULTY, PUBLIC }

public class Book{

    private final String isbn; private final String title; private final String author;

    public Book(String i,String t,String a){isbn=i;title=t;author=a;}

    public String getIsbn(){return isbn;} public String getTitle(){return title;}

    public String getAuthor(){return author;}

}

public class BookCopy{

    private final String copyId; private final Book book;

    private volatile BookStatus status=BookStatus.AVAILABLE;

    private volatile LocalDate dueDate=null;

    public BookCopy(String id,Book b){copyId=id;book=b;}

    public synchronized boolean checkout(int days){

        if(status!=BookStatus.AVAILABLE) return false;

        status=BookStatus.CHECKED_OUT; dueDate=LocalDate.now().plusDays(days); return true;

    }

    public synchronized void returnBook(){status=BookStatus.AVAILABLE;dueDate=null;}

    public BookStatus getStatus(){return status;}

    public LocalDate getDueDate(){return dueDate;}

    public String getCopyId(){return copyId;}

    public Book getBook(){return book;}

}

public interface SearchStrategy{List<Book> search(List<Book> books,String query);}

public class TitleSearch implements SearchStrategy{

    public List<Book> search(List<Book> b,String q){

        return

        b.stream().filter(bk->bk.getTitle().toLowerCase().contains(q.toLowerCase())).collect(Collectors.toList());

    }

}
```



```

}

public boolean checkout(String isbn,String memberId,int days){

return copies.stream()

.filter(c->c.getBook().getIsbn().equals(isbn)&&c.getStatus()==BookStatus.AVAILABLE)

.findFirst().map(c->c.checkout(days)).orElse(false);

}

public double returnBook(String copyId){

BookCopy copy=copies.stream().filter(c->c.getCopyId().equals(copyId))

.findFirst().orElseThrow(()->new IllegalArgumentException("Copy not found"));

LocalDate due=copy.getDueDate(); copy.returnBook();

eventBus.publish("AVAILABLE_"+copy.getBook().getIsbn(),copy.getBook());

long overdue=due!=null?Math.max(0,ChronoUnit.DAYS.between(due,LocalDate.now())):0;

return fineCalc.calculate(overdue,MemberType.PUBLIC);

}

public void reserveBook(String isbn,String memberId){

eventBus.subscribe("AVAILABLE_"+isbn,(t,d)->

System.out.println("Notifying "+memberId+": book "+isbn+" available"));

}

}

```

- + Strategy: SearchStrategy — swap search at runtime (title/author/ISBN)
- + Observer: EventBus notifies reserved members when book returned
- + Strategy: FineCalculator — different rates per MemberType
- + synchronized on BookCopy.checkout — concurrent checkout safe

Part 16: Interview Protocol + Timing + Scoring Rubric

The 45-Minute Framework

Time	Do This	Common Mistake
0-3 min	Clarify: scale? concurrency? which features in scope?	Starting to code immediately
3-5 min	Say entities out loud: "I see ParkingLot, Floor, Spot, Vehicle, Ticket"	Jumping to code before knowing your classes
5-8 min	Define interfaces + enums ONLY. No method body yet.	Writing implementation before contract
8-35 min	Implement core classes. Name pattern as you use it every time.	Silently coding with no explanation
35-40 min	Thread safety, extensibility, edge cases.	Stopping here — show failure mode thinking
40-45 min	"To add X: new class implementing Y. Zero existing changes. OCP."	Not discussing extensibility or testability

What to Skip (Mention Verbally)

Skip (say "I would add this")	Always Implement
All getters and setters	State-changing methods (park, reserve, withdraw)
Input validation strings	Throwing exceptions (just throw)
Logging statements	Thread safety (synchronized, volatile)
toString overrides	Core algorithm (SCAN, greedy settlement, LRU eviction)
Full main() demo	Pattern implementation (Strategy, Factory, State)
Edge case strings	Atomic-or-rollback on multi-step operations

Pattern Decision Guide — What to Say

You see...	Pattern	What to say out loud
One global instance	Singleton	I am using Singleton with DCL + volatile because...
Caller should not know concrete class	Factory	I am using Factory to centralise creation because...
Many optional params	Builder	Builder prevents telescoping constructors here because...
Multiple algorithms, swappable	Strategy	Strategy here — to add new algo, just new class, OCP
One event, many listeners	Observer	Observer decouples producer and consumers here

Add features without modifying	Decorator	Decorator stacks features at runtime without modifying
Object behaviour changes by state	State	State pattern — no if/else chains, each state handles events
Undo/redo needed	Command	Command encapsulates operation — enables undo/redo
Priority handler chain	Chain	Chain of Responsibility — each handler decides or passes
Fixed skeleton, varying steps	Template Method	Template Method prevents duplicate algorithm structure

Thread Safety Quick Reference

Situation	Use	Example in Problems
Method modifies shared field	synchronized on method	Seat.reserve(), ParkingSpot.park()
Map shared across threads	ConcurrentHashMap	activeTickets in ParkingLot
Simple flag, one writer	volatile boolean	shutdown flag, status field
Counter, high contention	AtomicInteger	request counter, retry count
Many readers, few writers	ReadWriteLock	LRU cache, config cache
Producer-consumer queue	BlockingQueue	job queue, notification queue
Per-thread state	ThreadLocal	DB connection per thread, request context

Red Flags — What Costs You the Offer

- Starting at implementation before identifying entities and interfaces
- God class: one class 400+ lines, does everything
- if/else chains where State or Strategy should be used
- Everything public — zero encapsulation
- Pattern name without justification: "I used factory" with no reason why
- Forgetting synchronized on shared mutable state
- "I would rewrite this to add X" — shows no OCP thinking

Green Flags — What Gets You Hired

- + Entities and relationships on paper before single line of code
- + Interfaces defined before implementations every time
- + "I am using Strategy here because we need to swap the pricing algorithm — OCP"
- + "synchronized on reserve() because two threads could reserve the same seat"
- + "I validate in constructor — fail fast on invalid state"
- + "To add a new payment type we just implement PaymentStrategy — zero existing changes"
- + "I would unit test by mocking PricingStrategy and injecting it — DIP makes this easy"

Scoring Rubric

Area	Weight	What They Check
OOP Correctness	25%	Encapsulation, correct inheritance, polymorphism
Design Patterns	25%	Right pattern for right problem. Named and justified.
Extensibility	20%	New feature = new class. OCP demonstrated naturally.
Thread Safety	15%	Shared state identified, correct mechanism used
Code Quality	15%	Readable names, validation, exceptions, no god classes

The 7/10 Readiness Test

Take any problem from Parts 7-15.

Set 40-minute timer. Blank file. No reference.

Grade yourself:

Stated entities before coding? Y/N

Defined interfaces before impl? Y/N

Named 2+ patterns with justification? Y/N

Added synchronized where needed? Y/N

Done in 40 minutes? Y/N

Can add new feature with new class only? Y/N

All 6 Y = 7/10. You are ready.

Any N = spend one more focused day on that gap.

Company-Specific Bars

Company	Bar	Focus Areas
Amazon	7/10	Working code + extensibility. LP: design something you are proud of.
Flipkart	7/10	Complete implementation. Thread safety. Full error handling.
Google	6.5/10	Testability above all. How would you unit test this?
Grab/Gojek	6.5/10	Practical patterns. Concurrency awareness.
Rubrik	8/10	Make it distributed. Consistent hashing extension.
Rippling	8/10	Speed. Complete in 35 min. Pattern naming crisp.